

CSS311 – Parallel and Distributed Computing

Lab 2: False Sharing, SISD/SIMD, and OpenMP Parallelism

Instructions

- Use at least 10^6 elements for arrays and 4096×4096 for matrices to ensure measurable timing differences.
-

Question 1: OpenMP – Pi Estimation

1. Write an OpenMP program with C/C++ that estimates the value of pi (π) using a following numerical integration formula called rectangle rule.

$$Area = \int_0^1 \frac{4}{1+x^2} dx = \pi$$

The following components are to be shown.

- (a) Write the sequential version program to estimate the value of pi (π). Test the result with classical integration value.
 - (b) Write the parallel version program to estimate the same. Test the result with classical integration value and by (a). It includes number of threads involved and the area calculated by which thread number.
 - (c) Identify the line of statement which leads the race condition. Race condition occurs when the multiple threads accessing a shared variable. If it exists, how will you handle this problem? Use appropriate OpenMP clause and find the solution. Test the result with classical integration value and by (a) and (b).
-

Question 2: False Sharing

Investigate the impact of false sharing on parallel performance using OpenMP.

- (a) Create an OpenMP program where each thread repeatedly updates its own counter stored in a **contiguous integer array** (i.e., all counters packed together in memory). Measure and display the execution time.
 - (b) Modify the program using a **padded structure** so that each thread's counter occupies a separate cache line (typically 64 bytes). Measure and display the execution time.
 - (c) Implement a third version using the **OpenMP reduction** clause. Compare all three versions namely false sharing, padded, and reduction in terms of execution time and explain the observed performance differences.
-

Question 3: SISD vs SIMD

Explore the effect of compiler auto-vectorization on array computation performance.

- (a) Write a program to perform **element-wise addition** of two large arrays (at least 10^6 elements). Compile using `-O0` (no optimization) and measure the execution time.
 - (b) Compile the same program using `-O3 -march=native`. Measure the execution time and compare with part (a). Explain the speedup.
 - (c) Generate the compiler vectorization report using `-fopt-info-vec-optimized`. Identify whether the loop is auto-vectorized and note which report lines confirm it.
 - (d) Modify the loop to introduce a **loop-carried dependency** (e.g., $A[i] = A[i-1] + B[i]$). Recompile with `-O3 -march=native` and check why vectorization fails. Confirm using the vectorization report.
 - (e) Measure the execution time of (a), for the different **scheduling policies** namely OpenMP with `schedule(static)`, OpenMP with `schedule(dynamic)` and OpenMP with `schedule(guided)`.
-

Question 4: OpenMP + SIMD

Combine thread-level and data-level parallelism and measure their combined contribution. Use either matrix addition or element-wise array multiplication. Choose one and apply it consistently across all sub-parts.

- (a) **Sequential:** Implement matrix addition **or** element-wise array multiplication as a sequential program. Measure the execution time.
 - (b) **OpenMP only:** Parallelize the same program using OpenMP (thread-level parallelism only). Measure the execution time and compute speedup over part (a).
 - (c) **SIMD only:** Compile the sequential version (part a) with `-O3 -march=native` to enable auto-vectorization. Measure the execution time and compute speedup over part (a).
 - (d) **OpenMP + SIMD combined:** Apply both OpenMP parallelism and SIMD optimization together. Measure the execution time and compute speedup over all previous versions.
 - (e) Prepare a **comparison table** with columns: Implementation | Execution Time | Speedup vs Sequential.
-